Zellic

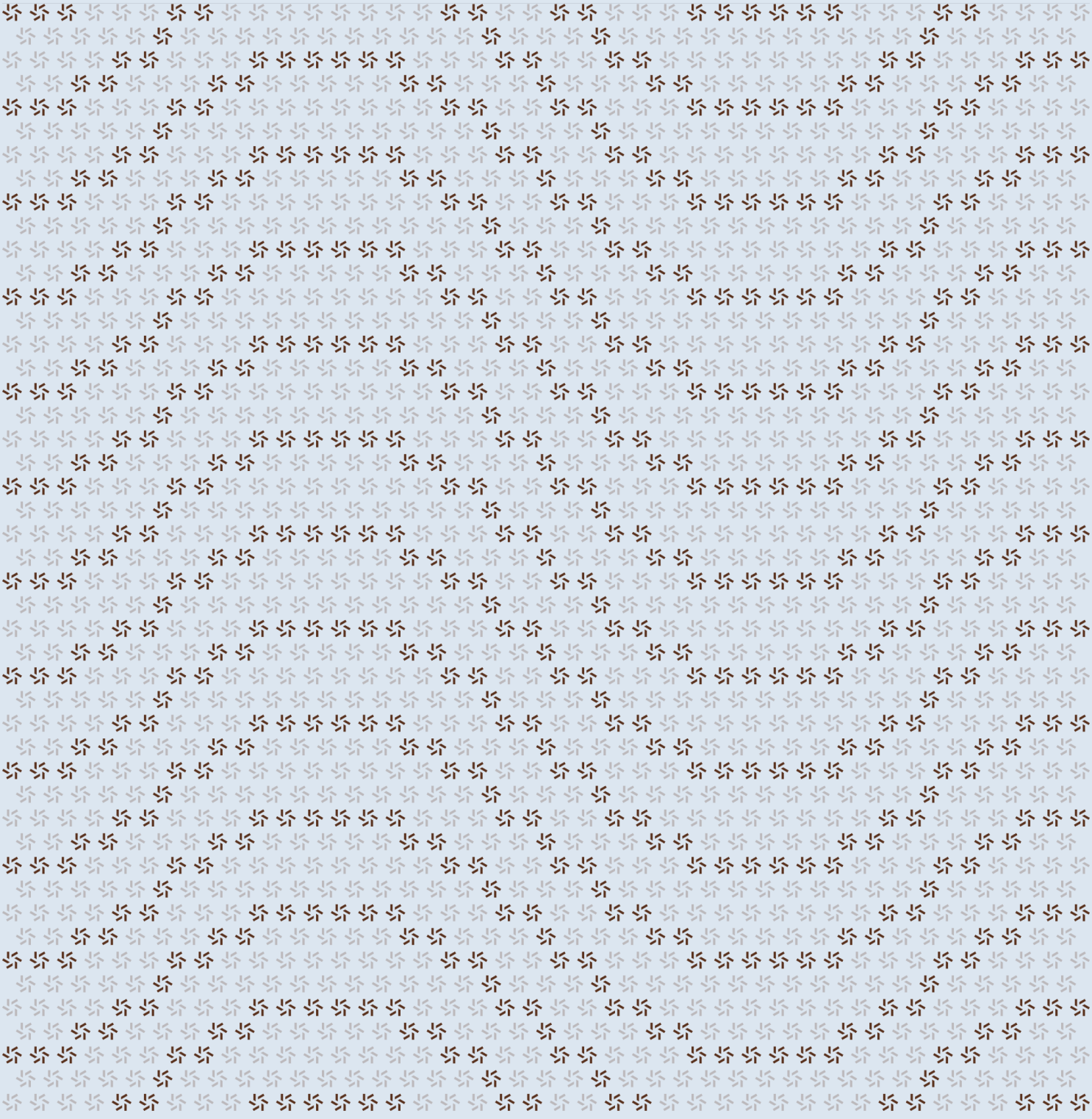
Prepared for
Stanford Liu
Initia Labs

Prepared by
Sunwoo Hwang
Varun Verma
Zellic

November 14, 2025

Initia Pull 226

Aptos Move Application Security Assessment



Contents

About Zellic **3**

1. Overview **3**

1.1. Executive Summary 4

1.2. Goals of the Assessment 4

1.3. Non-goals and Limitations 4

1.4. Results 4

2. Introduction **5**

2.1. About Initia Pull 226 6

2.2. Methodology 6

2.3. Scope 8

2.4. Project Overview 8

2.5. Project Timeline 9

3. Detailed Findings **9**

3.1. Large Single-Sided Liquidity Provisions Can Revert Due to Pow Base Range Check 10

4. Patch Review **11**

4.1. Notable changes 12

5. Assessment Results **12**

5.1. Disclaimer 13

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Initia Labs from November 12th to November 13th, 2025. During this engagement, Zellic reviewed Initia Pull 226's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is there any malfunction of liquidity calculations?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Initia Pull 226 modules, we discovered one finding, which was of low impact.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	0

2. Introduction

2.1. About Initia Pull 226

Initia Labs contributed the following description of Initia Pull 226:

> The InitiaDEX is a decentralized exchange (DEX) built on the Initia L1. It serves as a Balancer-style weighted pool for diverse asset pairs

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the modules.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Initia Pull 226 Modules

Type	Move
Platform	Aptos
Target	movevm
Repository	https://github.com/initia-labs/movevm ↗
Version	`16691ddec5aa058312581f7299ff1415f976e37a`
Programs	`precompile/modules/initia_stdlib/sources/dex.move`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of two person-days. The assessment was conducted by two consultants over the course of two calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Sunwoo Hwang
Engineer
sunwoo@zellic.io ↗

Varun Verma
Engineer
varun@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

November 12, 2025	Kick-off call
November 12, 2025	Start of primary review period
November 13, 2025	End of primary review period

3. Detailed Findings

3.1. Large Single-Sided Liquidity Provisions Can Revert Due to Pow Base Range Check

Target	initia_std::dex		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `calculate_provide_liquidity_return_amount` function uses the `pow` function to calculate invariant ratios for non-proportional liquidity provisions. The `pow` function has a hardcoded base range requirement of `0.01 < base < 2`.

When users attempt very large single-sided liquidity provisions (approximately >99x the current pool balance), the inverse balance ratio calculation drops below the 0.01 threshold:

```
let inv_coin_a_balance_ratio_without_fee =
    bigdecimal::from_ratio_u64(
        coin_a_amount,
        coin_a_amount + coin_a_amount_in_without_fee
    );
```

For example, if `coin_a_amount = 100` and `coin_a_amount_in_without_fee = 10,000`, the ratio becomes `100 / 10,100 ≈ 0.0099`, which is below the 0.01 minimum.

This causes the transaction to abort with `EOUT_OF_BASE_RANGE` error.

Impact

Users attempting to provide liquidity in amounts significantly larger than the existing pool (>99x) will have their transactions revert. This creates a practical ceiling on single-sided provision sizes, which may:

1. Confuse whale users or protocols attempting large deposits
2. Require multiple smaller transactions to achieve the desired liquidity provision
3. Impact integrations that expect large single-sided deposits to succeed

No funds are at risk, but UX is degraded for edge-case scenarios.

Recommendations

1. **Document the limitation**: Add comments and user-facing documentation explaining the maximum single-sided provision ratio (~99x pool size)

2. **Consider alternative approaches**:
    - Implement a pre-check that validates input sizes before executing expensive calculations
    - Add a helpful error message that suggests splitting large provisions

3. **Add input validation**:

```
let max_single_sided_ratio = bigdecimal::from_ratio_u64(99, 1);
assert!(
    bigdecimal::from_ratio_u64(amount_in, pool_amount) <=
    max_single_sided_ratio,
    error::invalid_argument(EEXCESSIVE_SINGLE_SIDED_PROVIDE)
);
```

Remediation

This issue has been acknowledged by Initia Labs.

4. Patch Review

This section documents notable and minor changes applied to the in-scope dex module in PR #226 ↗

4.1. Notable changes

The following were notable changes made to the codebase.

Pow base range check

The `pow` function has been updated to check the lower bound of the base. If the base is below the lower bound (0.01), the function will revert with a `EOUT_OF_BASE_RANGE` error.

Proportional amount calculation

A new proportional amount calculation step has been added before computing the invariant ratio. This logic handles cases where both `coin_a` and `coin_b` have non-zero amounts without fees.

The implementation:

1. Calculates balance ratios for both coins:
    - `coin_a_balance_ratio_without_fee = (coin_a_amount + coin_a_amount_in_without_fee) / coin_a_amount`
    - `coin_b_balance_ratio_without_fee = (coin_b_amount + coin_b_amount_in_without_fee) / coin_b_amount`
2. Selects the lower ratio and calculates proportional increase amounts:
    - If `coin_a_balance_ratio_without_fee < coin_b_balance_ratio_without_fee`, it uses `coin_a`'s ratio to calculate the proportional `coin_b_increase_amount`
    - Otherwise, it uses `coin_b`'s ratio to calculate the proportional `coin_a_increase_amount`
    - The calculated increase amounts are capped to not exceed the available amounts without fees
3. Applies the proportional amounts:
    - Updates `coin_a_amount` and `coin_b_amount` by adding the increase amounts
    - Reduces `coin_a_amount_in_without_fee` and `coin_b_amount_in_without_fee` by the same amounts
4. Updates the invariant ratio to the proportional increase ratio

This ensures that the proportional portion of the liquidity provision is handled first, with the remaining non-proportional amounts processed in subsequent calculations.

5. Assessment Results

During our assessment on the scoped Initia Pull 226 modules, we discovered one finding, which was of low impact.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.